

Lab5 Page Tables

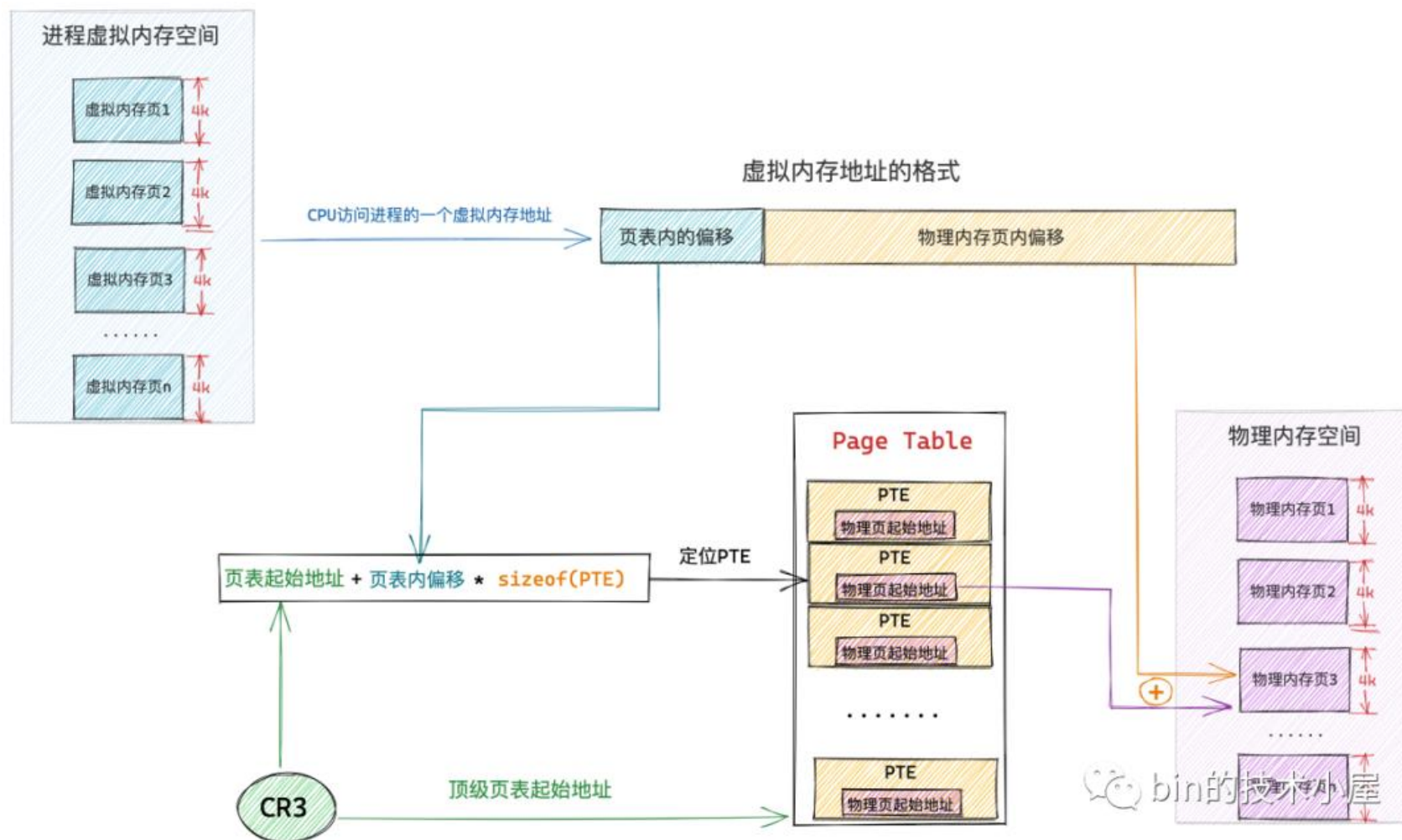
2026/5/8 王天择

wtz514@sjtu.edu.cn

Lab内容

- **Task 1** (*improve getpid()*). 通过修改页表的实现来加速 *getpid()* 系统调用。
 - 首先了解通过修改页表来加速系统调用的实验原理。
 - 实现一个用户态可读的 page, 避免切换到内核态才能进行 system call 读数据。
- **Task 2** (*print pgtbl*). 实现一个函数, 在启动 *xv6* 时, 打印出页表的内容。
 - 了解如何对一个页表进行遍历/递归。
- **Task 3** (*access detection*). 向 *xv6* 添加一个新特性, 报告哪些 *pages* 已经被访问。
 - 实现一个函数, 检查指定页表中的访问位, 并向用户空间返回访问信息。

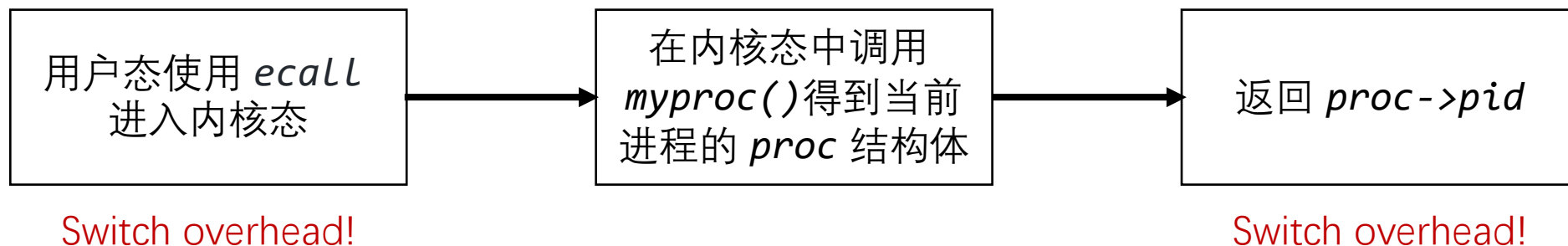
Review: 进程通过页表访问物理内存页的方法



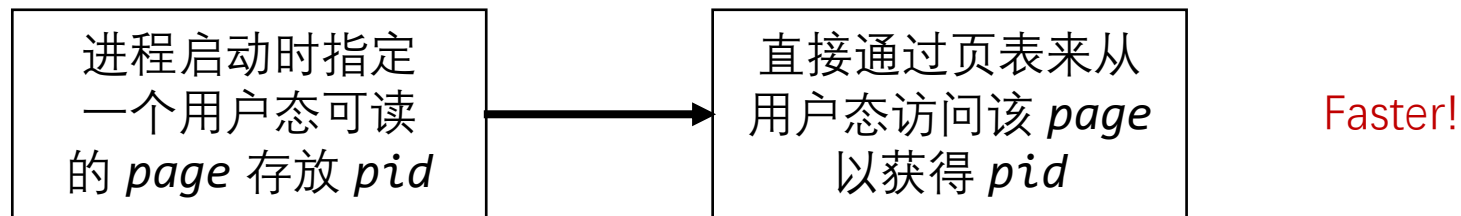
Task 1: 通过页表实现来加速 *getpid()* 系统调用

1. 实验原理:

➤ 传统的 *getpid()* 系统调用流程:



➤ 加速后的 *getpid()* 流程:



Task 1: 通过页表实现来加速 *getpid()* 系统调用

2. 实验提示:

- 可以使用 *kernel/proc.c* 中的 *proc_pagetable()* 来执行映射
- 选择只允许用户空间读取页面的权限位 *PTE_R* | *PTE_U*
- 在 *allocproc()* 中为进程分配和初始化该 *page*
- 使用 *mappages()* 函数来将物理地址和虚拟地址进行映射
- 在 *freeproc()* 中释放该页面
- 在 *kernel/proc.c* 中的 *proc_freepagetable()* 中取消 PTE 映射

Task 1: 通过页表实现来加速 *getpid()* 系统调用

3. 实验步骤:

➤ Step 1. 准备

当每一个进程被创建，映射一个只读的页在 *USYSCALL* （在*memlayout.h*定义的一个虚拟地址）处。存储一个 *struct usyscall* （定义在 *memlayout.h*）结构体在该页的开始处，并且初始化这个结构体来保存当前进程的 *PID*。这个 lab 中，*ugetpid()* 已经在用户空间给出，它将会使用 *USYSCALL* 这个映射。

```
#ifdef LAB_PGTBL
#define USYSCALL (TRAPFRAME - PGSIZE)

struct usyscall {
    int pid; // Process ID
};
#endif
```

Task 1: 通过页表实现来加速 *getpid()* 系统调用

3. 实验步骤:

- Step 2. 在 *kernel/proc.h* *proc* 结构体中添加一项指针来保存这个共享页面的地址

```
struct proc {  
    ...  
    struct usyscall *usyscallpage; // share page whithin kernel and user  
    ...  
}
```

Task 1: 通过页表实现来加速 *getpid()* 系统调用

3. 实验步骤:

- Step 3. 在 *kernel/proc.c* 的 *allocproc()* 中为其分配空间(*kalloc*), 并初始化其保存当前进程的PID

```
static struct proc*  
allocproc(void) {  
    ...  
    [   
  
    Insert here  
  
    p->usyscallpage->pid = p->pid;  
  
    // Set up new context to start executing at forkret,  
    // which returns to user space.  
    memset(&p->context, 0, sizeof(p->context));  
    p->context.ra = (uint64)forkret;  
    p->context.sp = p->kstack + PGSIZE;  
    ...  
}
```


Task 1: 通过页表实现来加速 *getpid()* 系统调用

3. 实验步骤:

- Step 4. 在 *kernel/proc.c* 的 *proc_pagetable(struct proc *p)* 中将这个映射 (*PTE*) 写入 *pagetable* 中, 权限是用户态可读

```
pagetable_t
proc_pagetable(struct proc *p) {
    ...

     Insert here

    // map the trampoline code (for system call return)
    // at the highest user virtual address.
    // only the supervisor uses it, on the way
    ...
}
```

Task 1: 通过页表实现来加速 *getpid()* 系统调用

3. 实验步骤:

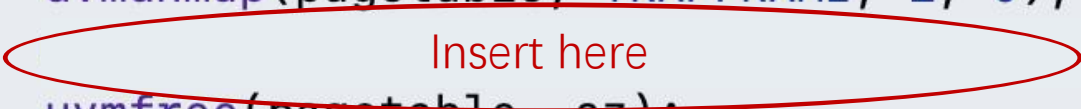
- Step 5. 确保释放进程的时候, 能够释放该共享页, 同样在 *kernel/proc.c* 中的 *freeproc(struct proc *p)*中实现

```
static void
freeproc(struct proc *p) {
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    // add start
    Insert here
    // add end
    if(p->pagetable)
        proc_freepagetable(p->pagetable, p->sz);
}
```

Task 1: 通过页表实现来加速 *getpid()* 系统调用

3. 实验步骤:

- Step 6. 在 *pagetable* 中仍然存在我们之前的 PTE 映射, 需要在 *kernel/proc.c* 的 *proc_freepagetable(pagetable_t pagetable, uint64 sz)* 中取消

```
void
proc_freepagetable(pagetable_t pagetable, uint64 sz) {
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    
    uvmfree(pagetable, sz);
}
```

运行 *./pgtbltest* 通过 *ugetpid_test* 即可 (*pgaccess_test* 不用管, 是 *task3*)。

Task 2: 在启动 *xv6* 时打印页表内容

1. 实验要求:

写一个函数来打印页表的内容。这个函数定义为 `vmprint()`。它应该接收一个 `pagetable_t` 类型的参数，并且按照下面的格式打印。在 `exec.c` 中的 `return argc` 之前插入 `if(p->pid==1) vmprint(p->pagetable)`，用来打印第一个进程的页表。

当你做完这些之后，运行 `qemu` 之后应该看到如下输出，它在第一个进程 `init` 完成之前打印出来。

```
page table 0x0000000087f6b000
..0: pte 0x0000000021fd9c01 pa 0x0000000087f67000
.. ..0: pte 0x0000000021fd9801 pa 0x0000000087f66000
.. .. ..0: pte 0x0000000021fda01b pa 0x0000000087f68000
.. .. ..1: pte 0x0000000021fd9417 pa 0x0000000087f65000
.. .. ..2: pte 0x0000000021fd9007 pa 0x0000000087f64000
.. .. ..3: pte 0x0000000021fd8c17 pa 0x0000000087f63000
..255: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..511: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..509: pte 0x0000000021fdcc13 pa 0x0000000087f73000
.. .. ..510: pte 0x0000000021fdd007 pa 0x0000000087f74000
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh
```

Task 2: 在启动 *xv6* 时打印页表内容

2. 实验提示:

- 可以在 *kernel/vm.c* 中实现 *vmprint()*
- 使用在 *kernel/riscv.h* 文件末尾的宏定义。
- 参考 *kernel/vm.c* 中函数 *freewalk* 的实现方法, 使用递归来遍历页表。
- 将函数 *vmprint* 的声明放到 *kernel/defs.h* 中, 以便可以在 *exec.c* 中调用它。
- 使用 *%p* 格式化打印64位十六进制的 *PTEs* 和地址。

```
// Recursively free page-table pages.
// All leaf mappings must already have been removed.
void
freewalk(pagetable_t pagetable)
{
    // there are 2^9 = 512 PTEs in a page table.
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        if((pte & PTE_V) && (pte & (PTE_R|PTE_W|PTE_X)) == 0){
            // this PTE points to a lower-level page table.
            uint64 child = PTE2PA(pte);
            freewalk((pagetable_t)child);
            pagetable[i] = 0;
        } else if(pte & PTE_V){
            panic("freewalk: leaf");
        }
    }
    kfree((void*)pagetable);
}
```

Task 2: 在启动 *xv6* 时打印页表内容

3. 实验步骤:

- 效仿 *freewalk()* 的递归方法实现 *vmprint()*，但是需要展示此页表的深度，这时我们可以另外设置一个静态变量来指示当前打印页的深度信息，如果需要进入下一级页表就将其加一，函数返回就减一。

```
void
vmprint(pagetable_t pagetable)
{
     Insert here
}
```

Task 3: 检查哪些页已经被访问

1. 实验原理及要求:

在一个 *Risc V Sv32 page table* 包含了 2^{10} 个 *PTEs*, 每一个 4 bytes. *Sv32 PTE* 的图示如下:

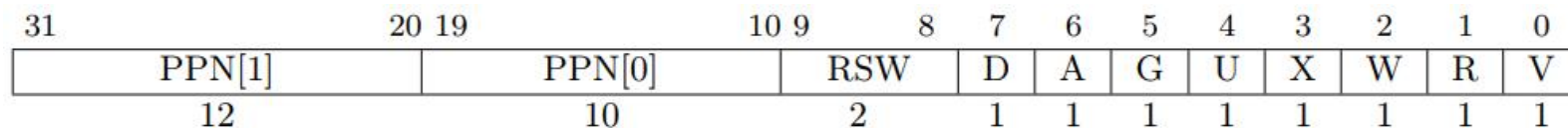


Figure 4.15: Sv32 page table entry.

每个 *Leaf PTE* 包含一个已访问 (A) 位。A 位表示自上次清除 A 位后, 虚拟页面已被读取、写入或获取。

这个实验中, 我们将实现一个系统调用 `sys_pgaccess()` 在文件 `kernel/sysproc.c` 中。这个系统调用会告诉我们哪一个页被访问过。此系统调用接收三个参数。第一: **被检查的第一个用户页的起始虚拟地址**; 第二: **被检查页面的数量**; 第三: **接收来自用户地址空间的一个 *buffer* 地址**, 将结果以掩码 (*bitmask*) 的形式写入。(*bitmask* 就是一个数据结构, 其一个位代表一个页面, 第一个页代表最低有效位)。

Task 3: 检查哪些页已经被访问

2. 实验提示:

1. 在 *user/pgtbltest.c* 的 *pgaccess_test()* 展示了如何使用 *pgaccess* 。
2. 使用 *argaddr()* 和 *argint()* 获取参数。
3. 对于要返回的 *bitmask* 值, 在 *kernel* 中建立临时的 *buffer*, 然后使用 *copyout* 拷贝到用户空间。
4. 可以限制可以被扫描的页的最大数量 (可选) 。
5. 在 *kernel/vm.c* 中的 *walk()* 很有用。它可以找到一个虚拟地址对应的 PTE, 返回其 *physical address*。
6. 需要在 *kernel/riscv.h* 中定义一个 *PTE_A*, 其为 *Risc V* 定义的 *access bit*。详细信息查看 [RISC-V privileged architecture manual](#)。
7. 检查完 *PTE_A* 位设置, 确保将其清零。因为如果不清零, 那么这些位都将会被设置为 1。因为检查其是否设置这个过程就访问了此页面, 在之后的过程中不能确定该页面 (被检查之前) 是否被访问过。
8. *vmprint()* 可能会在调试页表时很有用。

Task 3: 检查哪些页已经被访问

2. 实验提示:

1. 在 `user/pgtbltest.c` 的 `pgaccess_test()` 展示了如何使用 `pgaccess` 。

```
void
pgaccess_test()
{
    char *buf;
    unsigned int abits;
    printf("pgaccess_test starting\n");
    testname = "pgaccess_test";
    buf = malloc(32 * PGSIZE);
    if (pgaccess(buf, 32, &abits) < 0)
        err("pgaccess failed");
    buf[PGSIZE * 1] += 1;
    buf[PGSIZE * 2] += 1;
    buf[PGSIZE * 30] += 1;
    if (pgaccess(buf, 32, &abits) < 0)
        err("pgaccess failed");
    if (abits != ((1 << 1) | (1 << 2) | (1 << 30)))
        err("incorrect access bits set");
    free(buf);
    printf("pgaccess_test: OK\n");
}
```

*pgaccess*需要自己实现，自己定义返回被访问的页数总和，不单独实现

buf[1024], 访问第1页第一个字节 (最早是0页)

buf[2048], 访问第2页第一个字节 (最早是0页)

访问第30页第一个字节

pgaccess的效果就是设置abits，由于第1,2,30页被访问，所以abits的1,2,30位被设置为1

Task 3: 检查哪些页已经被访问

2. 实验提示:

- `walk()` 函数的使用非常重要, 它可以找到一个虚拟地址对应的 `PTE` 的地址。而我们就是需要检查 `PTE` 来判断其是否被访问过 (`PTE_A` 是否被设置)。
- 另一个重要的是要在检查之后清零此 `PTE_A` 位, 其在 `PTE` 的第 6 位 (从零开始)。我们通过如下方式来设置二进制位置的指定位置:

如何将一串二进制的制定位设置为指定的值。假如有一串二进制值 `1010`, 要将第二位设置为 `0`。

异或 ^

首先需要了解异或 `^` 运算符的概念和性质。和 0 异或等于它本身, 相同的值异或等于 1。另外一个值得注意的是, 一个数和同一个值异或两次等于它本身。

公式

公式: $x = ((x \& (1 \ll n)) \oplus x) \oplus (a \ll n)$ 。 `x` 为原值, `n` 为第 `n` 个值, `a` 为想要设置的值 (0或1)。

首先 `(x & (1 << n))` 的值为: 保留第 `n` 位原来的值, 其他位置零。再将此值与原值 `x` 异或, 得到一个值: 除了第 `n` 个值为零, 其他位置为原值。(这是因为, 与 `0` 异或的那一位为原值, 与相同值异或的那一位为 `0`)。然后此时, 再与 `(a << n)` 异或, 将第 `n` 位设置为 `a` (这是因为与 `0` 异或为其本身)。

Task 3: 检查哪些页已经被访问

3. 实验步骤:

- 要检查一个 *page* 是否被访问, 只需要检查 *PTE* 中的 *A bit* 是否被设置, 实现一个系统调用 *sys_pgaccess()* 在文件 *kernel/sysproc.c* 中。
- 此系统调用接收三个参数。第一: 被检查的第一个用户页的起始虚拟地址; 第二: 被检查页面的数量; 第三: 接收来自用户地址空间的一个 *buffer* 地址, 将结果以掩码 (*bitmask*) 的形式写入。

运行 `./pgtbltest` 通过 `pgaccess_test` 即可。

```
int
sys_pgaccess(void)
{

return 0;
}
```

Insert here

内核态函数

link

```
void
pgaccess_test()
{
char *buf;
unsigned int abits;
printf("pgaccess_test starting\n");
testname = "pgaccess_test";
buf = malloc(32 * PGSIZE);
if (pgaccess(buf, 32, &abits) < 0)
err("pgaccess failed");
}
```

用户态

```
sub entry {
my $name = shift;
print ".global $name\n";
print "${name}:\n";
print "li a7, SYS_${name}\n";
print "ecall\n";
print "ret\n";
}
```

compile